# Prototipo 1

## 1. Equipo

**Número del equipo:** 2E

**Integrantes:**

1. Daniel Alfonso Cely Infante
2. Juan Diego Rozo Álvarez
3. Tomás Alejandro Bermudez Guaqueta
4. Misael Jesús Florez Anave
5. Michael Sebastian Caicedo Rosero
6. Guillermo Moya Romero
7. William David Pinilla Ocampo
8. Iván David Molina Leguízamo

## 2. Sistema de software

**Nombre:** PoleBet

**Logo:**



**Descripción:** PoleBet es una plataforma de apuestas **en vivo** sobre carreras de Fórmula 1 con **tokens simulados** (sin dinero real). Mientras la carrera avanza, el sistema abre mercados cortos de pregunta Sí/No; los jugadores apuestan durante una ventana de 30 segundos, la carrera se observa 60 segundos más y el sistema liquida automáticamente con los datos reales de la carrera. Quien acierta suma 10 + monto × 0,5 puntos.

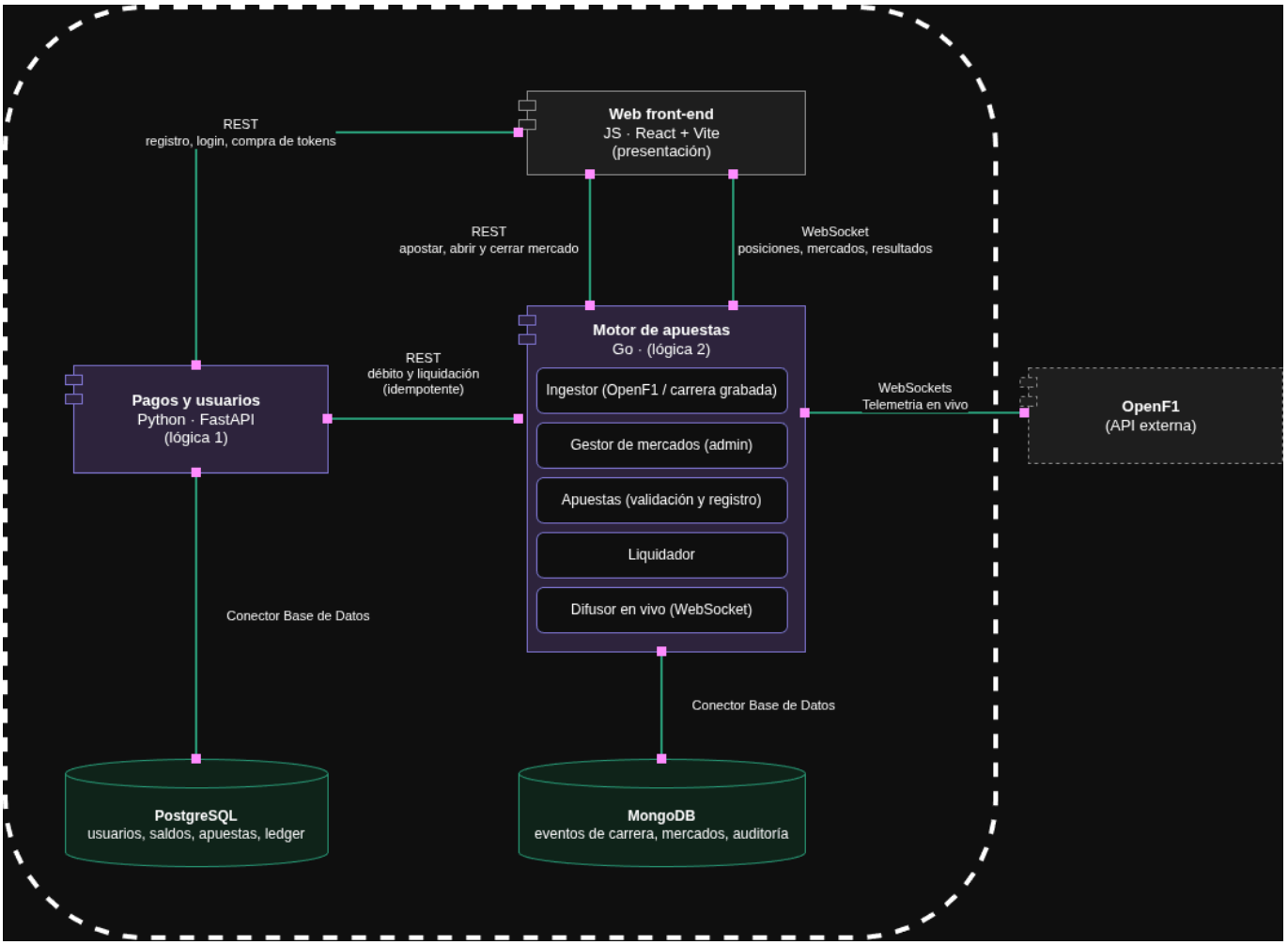
El **Prototipo 1** es un corte vertical que atraviesa todas las capas con un solo flujo:

1. Registro e inicio de sesión del jugador (JWT en cookie HttpOnly).
2. Compra simulada de tokens con tasa fija (1 token = 1.000 COP).
3. Ingesta de una carrera real (GP de Monza 2024) desde la API pública OpenF1 y su reproducción en tiempo real, con la tabla de posiciones en vivo por WebSocket.
4. Apertura de un mercado por el administrador: «¿Habrá algún cambio de posición durante la ventana de observación?».
5. Apuesta del jugador, liquidación automática y resultado enviado solo a ese jugador, sin recargar la página.

# 3. Estructuras arquitectónicas

## 3.1 Estructura de componentes y conectores (C&C)

### Vista C&C




### Estilo arquitectónico: SOFEA

La estructura se describe con una vista de **componentes y conectores (C&C)**: los componentes son las unidades que se ejecutan (front, servicios y bases de datos) y los conectores son los mecanismos por los que interactúan (REST, WebSocket y conectores de base de datos). Sobre esa vista, el estilo arquitectónico del sistema es **SOFEA (Service-Oriented Front-End Architecture)**.

**Cómo se aplica SOFEA en PoleBet:**

1. **La descarga de la aplicación está separada del intercambio de datos.** Nginx solo entrega una vez la SPA (HTML, JS y CSS estáticos compilados con Vite); a partir de ahí el navegador ya no pide páginas al servidor, solo datos.
2. **El flujo de presentación vive en el cliente.** La navegación entre pantallas, el estado de la sesión, la cuenta regresiva del mercado y la tabla de posiciones los maneja el front (React). Ningún servidor arma pantallas.
3. **El servidor solo expone servicios.** *Pagos y usuarios* y el *Motor de apuestas* no tienen lógica de presentación: ofrecen operaciones de negocio que el front consume directamente.
4. **El intercambio de datos es por contratos estándar.** JSON sobre **REST** para las acciones (registrarse, comprar, apostar, abrir mercado) y mensajes JSON {type, payload} sobre **WebSocket** para lo que el servidor empuja en vivo (posiciones, estado del mercado, resultado de la apuesta).

**Consecuencias en la vista C&C:**

- El **componente de presentación** (web front-end) es el cliente SOFEA y es el único que habla con el usuario.
- Los dos **componentes de lógica** son servicios independientes, cada uno dueño exclusivo de su **componente de datos** (PostgreSQL y MongoDB); entre ellos se comunican solo por un conector REST interno, nunca por una base compartida.
- El sistema resulta **distribuido y por niveles**: presentación, servicios y datos corren en procesos separados y solo se relacionan a través de conectores.

**Elementos**

- **Web front-end (presentación):** SPA en React + Vite servida por Nginx. Muestra registro, login, billetera, la carrera en vivo y el panel provisional del admin.
- **Pagos y usuarios (lógica 1):** servicio en Python/FastAPI. Maneja usuarios, login con JWT, saldo de tokens, débito de apuestas y puntos.
- **Motor de apuestas (lógica 2):** servicio en Go con cinco subcomponentes:
  - **Ingestor:** trae la carrera de OpenF1, la guarda y la reproduce en tiempo real.
  - **Gestor de mercados:** abre el mercado (admin) y lo cierra con el reloj del servidor.
  - **Apuestas:** valida cada apuesta y pide el débito a Pagos.
  - **Liquidador:** decide Sí/No al terminar la observación y liquida en Pagos.
  - **Difusor en vivo:** envía posiciones, mercados y resultados por WebSocket.
- **PostgreSQL (datos relacional):** usuarios, saldos, apuestas y ledger.
- **MongoDB (datos NoSQL):** eventos de carrera, mercados y contexto de apuestas.
- **OpenF1 (sistema externo):** API pública con los datos reales de la carrera, fuera de la frontera del sistema.

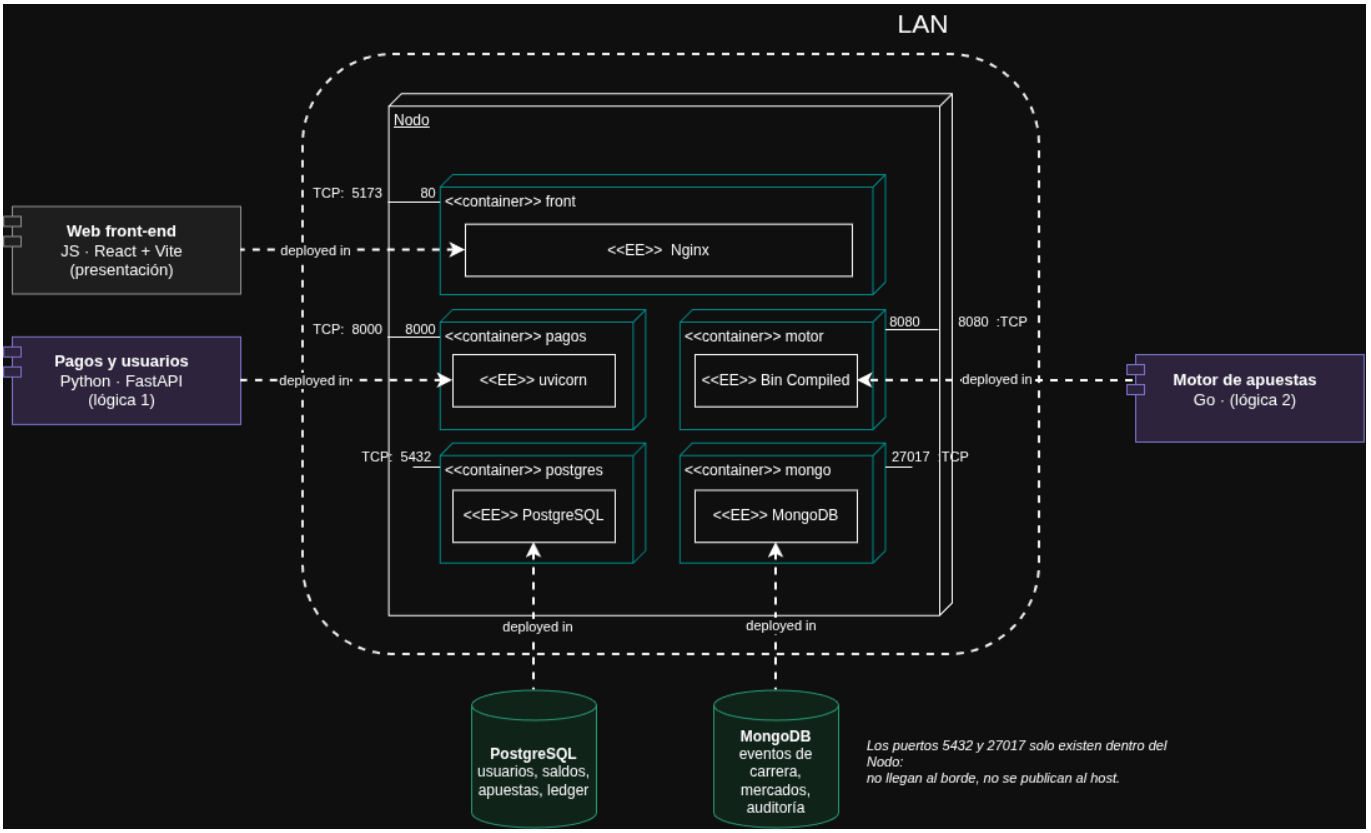
**Relaciones (conectores)**

- **Front → Pagos (REST):** registro, login y compra de tokens.
- **Front → Motor (REST):** apostar, abrir mercado y consultar el mercado actual y las posiciones.
- **Motor → Front (WebSocket):** posiciones en vivo, estado del mercado y resultado de cada apuesta.
- **Motor → Pagos (REST interno):** débito de la apuesta y liquidación de puntos, idempotentes.
- **Motor → OpenF1 (REST/HTTPS):** descarga de posiciones y pilotos de la carrera.
- **Pagos → PostgreSQL (conector de base de datos):** consultas y transacciones SQL.
- **Motor → MongoDB (conector de base de datos):** lectura y escritura de carrera, mercados y apuestas.

Cumplimiento de los requisitos no funcionales: arquitectura distribuida; 1 componente de presentación; 2 de lógica; 2 de datos (relacional y NoSQL); 2 tipos de conectores HTTP (**REST** y **WebSocket**); 3 lenguajes de propósito general (**Go**, **Python** y **TypeScript/JavaScript**); despliegue orientado a contenedores (sección 3.2).

## 3.2 Estructura de despliegue

**Vista de despliegue**




**Descripción**

Todo el sistema se despliega en **un solo nodo** (la máquina host) con Docker Engine, mediante Docker Compose. Cada componente de la vista C&C se despliega en **su propio contenedor**, y todos comparten una red interna tipo *bridge* (arqui_net) en la que se encuentran por nombre (por ejemplo, el Motor llama a http://pagos:8000).

| Contenedor | Componente desplegado | Entorno de ejecución | Puerto host → contenedor |
|---|---|---|---|
| front | Web front-end | Nginx (sirve el build estático de Vite) | TCP 5173 → 80 |

| Contenedor | Componente desplegado | Entorno de ejecución | Puerto host → contenedor |
|---|---|---|---|
| pagos | Pagos y usuarios | Python 3.12 + Uvicorn | TCP 8000 → 8000 |
| motor | Motor de apuestas | Binario compilado de Go (Alpine, usuario sin privilegios) | TCP 8080 → 8080 |
| postgres | PostgreSQL | PostgreSQL 17 | solo interno, TCP 5432 |
| mongo | MongoDB | MongoDB 7 | solo interno, TCP 27017 |

- **Solo el front y las dos APIs se publican al host**; los puertos de las bases de datos (5432 y 27017) existen únicamente dentro del nodo, así nadie fuera del sistema llega directo a los datos.
- **Orden de arranque con *healthchecks***: Pagos espera a que PostgreSQL esté sano; el Motor espera a MongoDB y a Pagos.
- **Persistencia** en volúmenes de Docker (postgres_data, mongo_data); MongoDB se inicializa con scripts que crean las colecciones, sus validadores y la carrera de prueba.
- **Configuración** centralizada en un archivo .env (puertos, credenciales, secreto del JWT, token interno entre servicios y ventanas de 30 s y 60 s).
- OpenF1 está fuera del nodo: el Motor sale a Internet por HTTPS solo durante la ingesta de la carrera.

# 4. Prototipo

Instrucciones para Linux. Repositorio: github.com/jdrsajonia/polebets

Requisitos: Docker Engine con el plugin Docker Compose v2 (docker compose version), git y curl; puertos 5173, 8000 y 8080 libres; conexión a Internet (para construir las imágenes y para la ingesta desde OpenF1). Si tu usuario no está en el grupo docker, antepón sudo a cada comando docker.

## 4.1 Descargar y configurar

```
git clone https://github.com/jdrsajonia/polebets.git
cd polebets
cp .env.example .env
```

El archivo .env ya trae valores válidos para desarrollo local; no hay que editar nada.

## 4.2 Levantar el sistema

```
ocker compose up --build -d
docker compose ps
```

Espera a que pagos y motor aparezcan como healthy (la primera vez tarda unos minutos mientras se construyen las imágenes). Para comprobarlo:

```
curl http://localhost:8000/health
curl http://localhost:8080/health
```

### 4.3 Cargar y reproducir la carrera

La ingesta y la reproducción se disparan por consola con un JWT de administrador. Ejecuta los tres comandos **en la misma terminal**:

```
# 1. JWT de administrador firmado con el JWT_SECRET del contenedor de Pagos
TOKEN=$(docker exec arqui_pagos python -c "import os, jwt, datetime;
print(jwt.encode({'sub': '00000000-0000-0000-0000-000000000001', 'rol':
'admin', 'exp': datetime.datetime.now(datetime.UTC) +
datetime.timedelta(hours=1)}, os.environ['JWT_SECRET'],
algorithm='HS256'))")

# 2. Ingesta del GP de Monza 2024 desde OpenF1 (una sola vez). Respuesta:
{"insertados":483}
curl -X POST http://localhost:8080/ingest/openf1 \
  -H "Cookie: session=$TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"carrera_id":"2024_monza","session_key":9590}'

# 3. Reproducción en tiempo real desde un punto aleatorio de la carrera
(dura hasta 30 min)
curl -X POST http://localhost:8080/ingest/replay \
  -H "Cookie: session=$TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"carrera_id":"2024_monza","factor":1}'
```

La reproducción debe estar corriendo **antes** de abrir un mercado, y no se debe relanzar mientras haya un mercado abierto o en observación.

### 4.4 Probar el flujo

1. Abre **http://localhost:5173** (usa localhost, no 127.0.0.1, para que la cookie de sesión llegue a los dos servicios).
2. En /registro crea un jugador e inicia sesión.
3. En **Billetera** compra tokens (montos múltiplos de 1.000 COP).
4. En otra ventana de incógnito o en otro navegador, inicia sesión como administrador con admin@arqui.local / cambiar-esta-clave-solo-dev (valores del .env). Te lleva a /admin: pulsa **Abrir mercado**.
5. Como jugador, en **Carrera** elige Sí o No, un monto y apuesta antes de que termine la cuenta regresiva de 30 s.
6. Unos 90 s después de abrir el mercado aparece el resultado: ganaste o perdiste, los puntos obtenidos y el puntaje total.

4.5 Detener y borrar los volúmenes

```
docker compose down -v
```